

챗 11. 커넥트HR 에이전트의 완성. 파이프라인과 근거, 그리고 마지막 여정

이번 챗이 끝나면

- 챗 8~10에서 만든 함수들을 하나의 파이프라인으로 조립합니다
- 답변에 근거를 붙여 줍니다. 정형 질문은 JSON, 비정형·복합 질문은 문서 페이지 이미지
- 챗 7의 채팅 UI를 재사용해서 실제 사용자 경험으로 완성합니다
- 챗 1~10의 모든 박스가 채워진 커넥트HR 에이전트의 전체 구성도를 확인합니다

준비하기. 실습 시작 전 한 번만 설정

1. 실습 폴더 이동

터미널 폴더 이동

```
cd rag-start/ex11
```

파일 구조는 다음과 같습니다.

ex11 디렉토리

```
ex11/
├─ run.py                # [참고] FastAPI 서버 실행 (챗 7 UI 재사용)
├─ app/                  # [참고] 채팅 API + DB
│   ├── main.py
│   ├── chat_api.py      # [실습] 근거 분기 로직 추가
│   └─ database.py
├─ src/
│   ├── pipeline.py      # [실습] Application Layer. RagPipeline 조립
│   ├── evidence_capture.py # [실습] PDF 페이지 → PNG 캡처 근거
│   ├── evidence.py      # [참고] 출처 추출
│   └─ agent.py          # [참고] IntegratedAgent (챗 6)
```

```

| | | └─ bm25_retriever.py      # BM25Retriever - rank-bm25 래퍼
| | | └─ ensemble_retriever.py # EnsembleRetriever - BM25 + ChromaDB 하이브리드
| | | └─ reranker.py           # CrossEncoderReranker (+ SimpleReranker 플백)
| | | └─ parent_doc.py         # ParentDocumentRetriever - 자식 → 부모 복원 (챗터 9)
| | |   └─ hybrid_parser.py     # 챗터 10 파서 이식 - 인텍싱 시 pypdf → Vision
| | |   └─ tools/               # [참고] Infrastructure Layer. SQL 3종 + ChromaDB 검색
| | |     └─ leave_balance.py
| | |     └─ sales_sum.py
| | |     └─ list_employees.py
| | |     └─ search_documents.py
| └─ templates/chat.html      # [실습] 근거 이미지·JSON 표시 영역 추가
| └─ static/
|   └─ evidence/              # 런타임 생성 (PDF 캡처 PNG 캐시)
|   └─ data/
|     └─ docs/                 # [참고] 원본 PDF
|     └─ chroma_db/            # 런타임 생성
|     └─ test_questions.json   # [참고]

```

2. 실습 환경 구축

터미널 환경 구성. macOS / Linux

```

cd ex11
python3.12 -m venv .venv
source .venv/bin/activate
cp .env.example .env
ollama pull llama3.1:8b
ollama pull qwen2.5vl:7b
pip install -r requirements.txt

```

터미널 환경 구성. Windows

```

cd ex11
py -3.12 -m venv .venv
.venv\Scripts\activate
copy .env.example .env
ollama pull llama3.1:8b
ollama pull qwen2.5vl:7b
pip install -r requirements.txt

```

3. 실습 순서

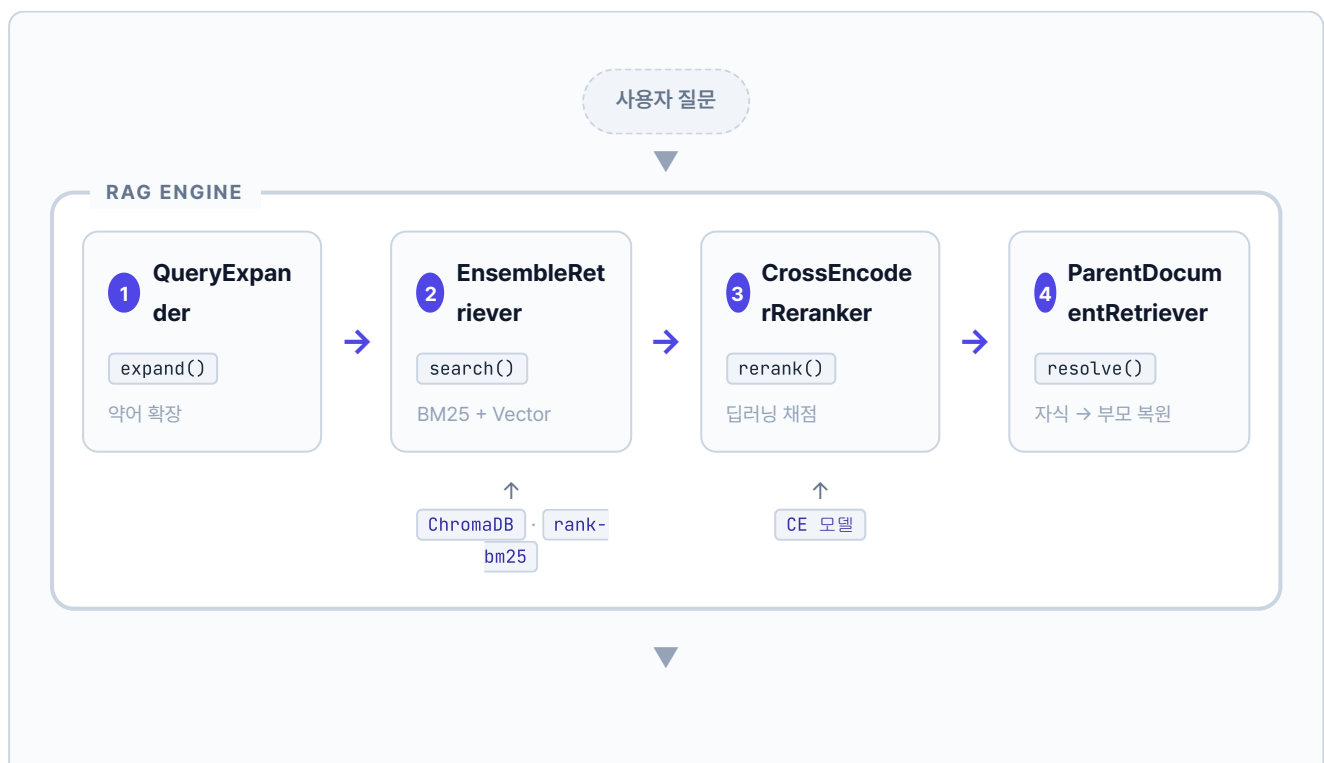
1. `src/pipeline.py` (D 조합: Semantic 청킹, 하이브리드 검색, 리랭커, 약어 확장, Parent Doc, Vision 파서)을 하나로 조립합니다
2. `src/evidence_capture.py` . PDF 페이지를 PNG로 렌더링해 근거로 제공합니다
3. `app/chat_api.py` + `templates/chat.html` . 정형(JSON)과 비정형(이미지) 근거 분기를 붙입니다

11.1 파이프라인 조립 - 튜닝 부품을 한 파일에 엮는다

팀장이 옆자리로 의자를 밀고 왔습니다. 모니터 한 칸엔 챕터 8~10에서 돌린 실험 결과가 여전히 떠 있었습니다.

팀장 : "이번엔 새 이론을 엮지 말고, 지금까지 본 튜닝을 한 자리에 모아 이어 붙이기만 해요."

오픈이가 에디터에서 새 파일을 열었습니다. `pipeline.py`. 키보드 위에 손을 올린 채 빈 화면을 잠시 바라봤습니다. 이번 장의 할 일은 지금까지 따로 실험하던 **튜닝 부품**을 한 파일에 모아 질의 흐름 하나로 엮는 것입니다. 챕터 8·9에서는 조각별로 돌려 성능을 비교만 했다면, 여기서는 같은 개념을 함수로 `pipeline.py` 안에 모아 `run_pipeline` 하나로 차례차례 호출합니다.



최초 실행 시 `ex11/data/docs`를 하이브리드 파서(pypdf → Vision)로 인덱싱해 **자식 청크는 ChromaDB에**, **부모 페이지는 parent_docs.json에** 나눠 적재합니다. 네 부품은 질문이 들어올 때마다 자식 검색 → 부모 복원 순으로 실행됩니다

그림 11-1. RAG 엔진. 질문이 들어와 답변이 나가기까지

파이프라인을 **레이어드 아키텍처**로 분리합니다. `pipeline.py`의 `RagPipeline`이 질의가 들어올 때마다 네 부품

(`QueryExpander` · `EnsembleRetriever` · `CrossEncoderReranker` · `ParentDocumentRetriever`)

을 차례대로 부르고, 그 부품들은 `src/tools/`의 ChromaDB·BM25 같은 외부 저장소를 끌어 씁니다.

벡터DB는 `ex11/data/docs`를 자체 인덱싱해 **자식·부모 두 저장소**로 빌드합니다. 페이지 단위 원문은

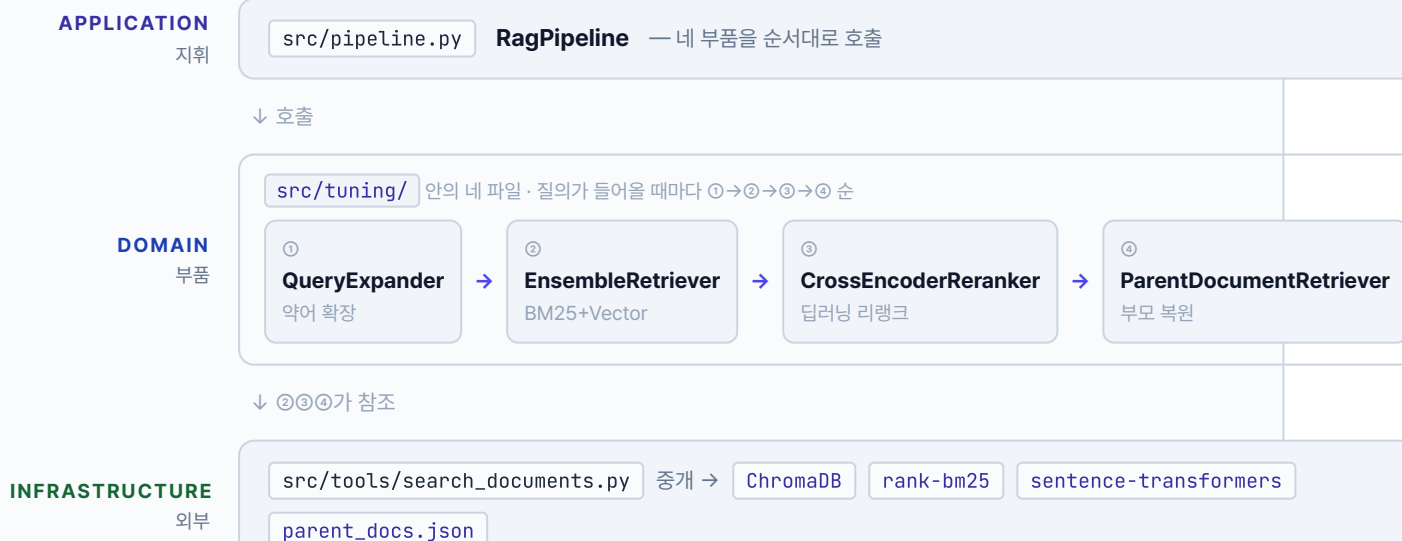
부모로 `ex11/data/parent_docs.json`에 두고, 220자 정도로 잘게 쪼갠 자식 청크는

ChromaDB에 `parent_id` 메타데이터와 함께 적재합니다. PDF는

`src/tuning/hybrid_parser.py`(챗터 10의 하이브리드 파서를 이식한 모듈)가 pypdf로 먼저 꺼내 보고 스캔본이면 Vision LLM으로 넘어가 텍스트를 뽑습니다. BM25 인덱스와 Cross-Encoder 모델은

`EnsembleRetriever` · `CrossEncoderReranker` 객체가 최초 사용 시 1회만 초기화합니다.

레이어드 아키텍처. 위에서 아래로 호출·참조만 흐릅니다



세 층이 단방향으로만 얹히기 때문에(위→아래), 한 층의 내부를 바꿔도 다른 층은 영향이 적습니다. 예) 벡터DB를 Qdrant로 갈아끼울 땐 Infrastructure만 고치면 됩니다

`src/pipeline.py`를 열고 TODO의 `pass`를 지우고 아래 코드를 작성합니다.

실습 1 `src/pipeline.py`. RagPipeline 조립 (Application Layer)

```
from .tuning import (
    CrossEncoderReranker,
    EnsembleRetriever,
    ParentDocumentRetriever,
    QueryExpander,
)

class RagPipeline:
    # TODO: 4개 도메인 객체를 생성자에서 준비하고 run()에서 순서대로 호출합니다.
    def __init__(self, alpha: float = 0.5) → None:
        from .tools.search_documents import _get_store
        collection, parent_docs = _get_store()
        # 1. 약어 확장 (챕터 9)
        self.expander = QueryExpander()
        # 2. BM25 + ChromaDB 하이브리드 검색 (챕터 8) - 자식 청크 대상
        self.retriever = EnsembleRetriever(collection, alpha=alpha) if collection else None
        # 3. Cross-Encoder 리랭크 (챕터 8)
        self.reranker = CrossEncoderReranker()
        # 4. 자식 → 부모 복원, 중복 parent_id 제거 (챕터 9)
        self.parent_retriever = (
            ParentDocumentRetriever(child_collection=collection, parent_docs=parent_docs)
            if collection and parent_docs else None
        )

    def run(self, query: str, k: int = 3) → list[dict]:
        if self.retriever is None or self.parent_retriever is None:
            return []
        expanded = self.expander.expand(query)
        retrieved = self.retriever.search(expanded, k=k) # 자식 청크
        if not retrieved:
            return []
        reranked = self.reranker.rerank(expanded, retrieved, top_k=k * 2)
        parents = self.parent_retriever.resolve(reranked, top_k=k) # 부모 복원
        return parents[:k]
```

`RagPipeline.__init__`은 생성자 주입 패턴으로 도메인 객체를 한 번 준비해 두고, `run()`은 다섯 줄 안에서 확장 → 자식 검색 → 리랭크 → 부모 복원 → 절단을 호출합니다. 객체별 내부 구현은

`src/tuning/` 각 파일에 나뉘어 있어 개별 교체·테스트가 쉽습니다.

도메인 객체 (tuning/*)	역할
<code>QueryExpander.expand(q)</code>	<code>WFH</code> → <code>WFH(재택근무)</code> 사내 약어 확장
<code>EnsembleRetriever.search(q, k)</code>	BM25 + ChromaDB(Vector)로 자식 청크 검색. 두 점수 정규화 후 가중 합산
<code>BM25Retriever.search(q, k)</code>	<code>rank-bm25</code> 키워드 검색기 (EnsembleRetriever 내부에서 사용)
<code>CrossEncoderReranker.rerank(q, docs, k)</code>	<code>BAAI/bge-reranker-v2-m3</code> 딥러닝 모델로 쿼리·자식 청크 정합성 채점
<code>ParentDocumentRetriever.resolve(docs, k)</code>	자식 청크의 <code>parent_id</code> 로 부모 페이지 원문 복원. 같은 부모 중복 제거

11.2 근거 모양을 질문에 맞춘다

파이프라인이 답변을 돌려줄 때 어느 문서에서 가져왔는지 근거까지 같이 보여 주어야 독자가 믿습니다. 검색 결과에는 `(source, page)` 쌍이 이미 붙어 있으니, 그 정보로 PDF의 해당 페이지를 PNG로 떠 주면 답변 옆에 바로 붙일 수 있습니다. `src/evidence_capture.py`를 열고 TODO의 `pass`를 지우고 아래 코드를 작성합니다.

실습 1 `src/evidence_capture.py`. PDF 페이지 → PNG 렌더 + 캐시

```
import fitz # PyMuPDF
from pathlib import Path

BASE_DIR = Path(__file__).resolve().parent.parent
DOCS_DIR = BASE_DIR / "data" / "docs"
CACHE_DIR = BASE_DIR / "static" / "evidence"

def render_evidence_image(source_name: str, page_num: int, dpi: int = 150) → str:
    # TODO: PDF 특정 페이지를 PNG로 렌더링하고 /static/evidence/*.png URL 반환.
    # 1. data/docs/ 하위에서 PDF 경로 탐색
    pdf_path = next(DOCS_DIR.rglob(f"{source_name}.pdf"), None)
    if not pdf_path:
        return ""
```

```
# 2. 캐시 디렉토리 준비
CACHE_DIR.mkdir(parents=True, exist_ok=True)
out_path = CACHE_DIR / f"{source_name}_p{page_num}.png"

# 3. 캐시 적중 시 재사용
if out_path.exists():
    return f"/static/evidence/{out_path.name}"

# 4. PDF 열어서 해당 페이지 → PNG 저장
doc = fitz.open(str(pdf_path))
pix = doc[page_num - 1].get_pixmap(dpi=dpi)
pix.save(str(out_path))
doc.close()
return f"/static/evidence/{out_path.name}"
```

매개변수를 보면 `source_name`은 `test_questions.json`의 `relevant_sources`와 같은 PDF 파일명(확장자 제외), `page_num`은 1부터 시작하는 페이지 번호입니다. `dpi`가 클수록 선명하지만 파일도 커집니다. 150이 화면 표시용으로는 충분합니다.

`/static/evidence/` 경로는 FastAPI가 정적 파일로 이미 서빙하고 있어서, 반환한 URL을 그대로 ``에 쓰면 브라우저에 뜹니다. 캐시 폴더에 한 번 저장된 이미지는 다음 호출에서 재사용되니 성능 부담도 없습니다.

그런데 모든 질문에 PDF 이미지가 근거로 어울리는 건 아닙니다.

동료: "서약서나 취업규칙은 이렇게 이미지로 보는 게 좋은데, '1분기 매출 얼마야' 같은 질문은 이미지보다 숫자 그 자체가 더 편해요. 표라면 표, JSON이면 JSON으로 보여 주는 게 낫겠어요."

맞는 말이네. 비정형 답은 문서가 근거지만, 정형 답은 수치가 근거다.

질문의 성격에 따라 **근거의 모양**을 달리해야 합니다. 간단한 규칙으로도 충분합니다. 질문에 숫자·금액·수치·퍼센트 같은 단어가 있으면 **정형**, 그 외는 **비정형**. 정형이면 DB 조회 결과를 JSON 그대로 내보내고, 비정형이면 방금 만든 `render_evidence_image()`로 PDF 캡처 이미지를 붙입니다.

실무에서는 LLM에 "이 질문이 정형이냐 비정형이냐?"를 먼저 묻는 방식도 쓰지만, 호출이 한 번 더 늘어납니다. 이 책에서는 키워드 규칙으로 시작하고, 필요해지면 LLM 분류로 업그레이드하는 쪽으로 가겠습니다.

`app/chat_api.py`를 열고 TODO의 `pass`를 지우고 아래 코드를 작성합니다.

실습 2 app/chat_api.py. 카테고리 분류 + 근거 모양 분기

```
import re

_STRUCTURED_KEYWORDS = [
    "얼마", "몇", "금액", "매출", "수치", "비율", "%", "퍼센트",
    "명", "건", "개수", "총", "평균", "합계", "남은", "남아",
    "언제", "며칠", "시간", "분기", "연도", "년도",
]

_STRUCTURED_RE = re.compile(r"\b\d+[\.%가-힐]?")

def _classify_query_category(query: str) → str:
    # TODO: 질문을 정형(structured) / 비정형(unstructured)로 분류합니다.
    # 1. 숫자 토큰이 있는지
    if _STRUCTURED_RE.search(query):
        return "structured"
    # 2. 정형 키워드가 하나라도 포함되는지
    lower = query.lower()
    for kw in _STRUCTURED_KEYWORDS:
        if kw in lower:
            return "structured"
    # 3. 그 외는 비정형
    return "unstructured"
```

분류 결과는 `ChatResponse.category` 필드로 같이 내려보냅니다. 그리고 기존 엔드포인트 로직에서:

- `category = "structured"` 이면 DB 조회 결과를 `evidence_json` 필드에 담는다
- `category = "unstructured"` 이고 아직 `evidence_images` 가 비어 있으면, 상위 문서의 (source, page)로 `render_evidence_image()` 를 불러 이미지를 첨부한다

두 갈래가 한 엔드포인트 안에서 자연스럽게 갈라집니다.

정형 질문 → JSON

Q. 영업부 11월 매출 얼마야?

영업부 11월 매출은 1,240,500,000원입니다.

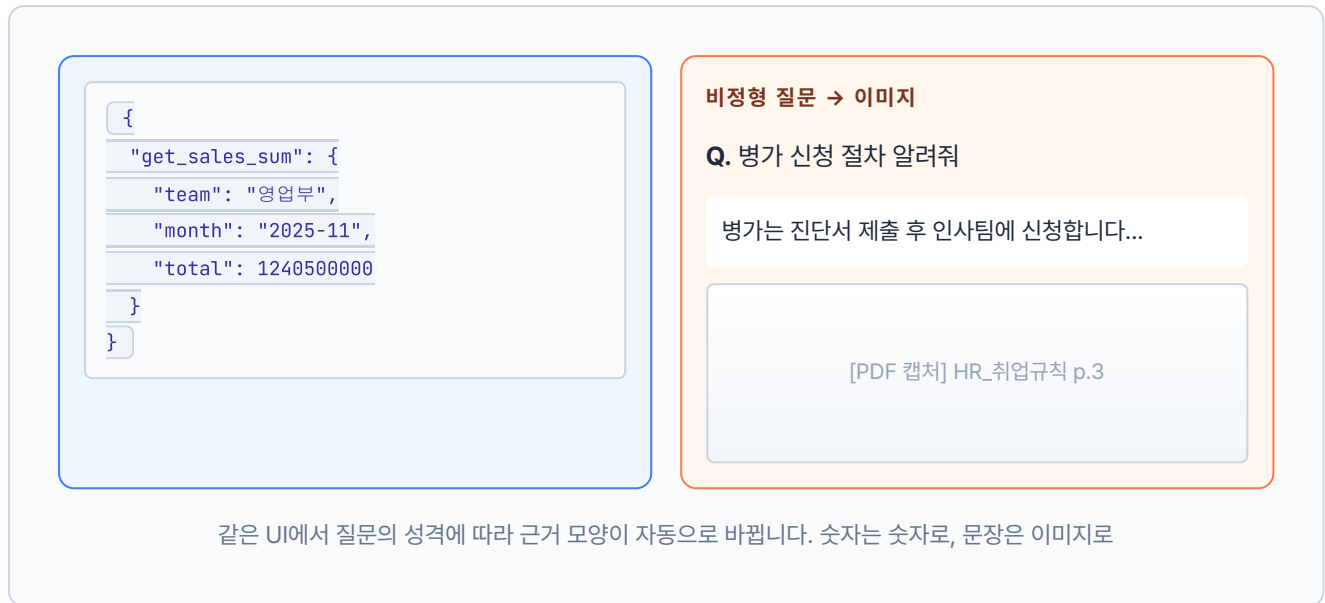


그림 11-2. 카테고리에 따라 갈라지는 근거 모양

동료 : "이제 매출은 매출답게, 규정은 규정답게 보여지네요."

팀장 : "좋아요. 이걸로 직원들이 '믿고 쓸 수 있는' 에이전트가 됐어요."

근거의 모양이 곧 답변의 신뢰다.

11.3 웹 UI로 확인한다

조립이 끝났습니다. 선택한 D 조합(Semantic 청킹, 하이브리드 검색, 리랭킹, 약어 확장, Parent Doc, Vision 파서)을 엮은 파이프라인으로 챗터 7의 채팅 UI를 다시 띄워 보겠습니다.

터미널 ex11 서버 실행 (챗터 7 UI + 챗터 8~10 튜닝 적용)

```
python run.py
```

서버 터미널에 기동 순서가 세 단계로 찍힙니다. 벡터DB 준비 → 리랭커 로드 → uvicorn 기동. 여기까지 오는 데 평균 2분이 걸립니다(최초 1회는 모델 다운로드로 더 걸립니다).



그림 11-3. 서버 기동 시 터미널에 찍히는 세 단계

브라우저에서 `http://localhost:8000/chat` 을 엽니다. 챕터 7의 ConnectHRAgent 채팅 UI가 뜹니다. 같은 UI, 다른 엔진.

오픈이가 채팅창으로 손을 옮겼습니다. 커서가 입력란에서 깜빡였습니다.

첫 질문. `병가 신청 절차 알려줘` .

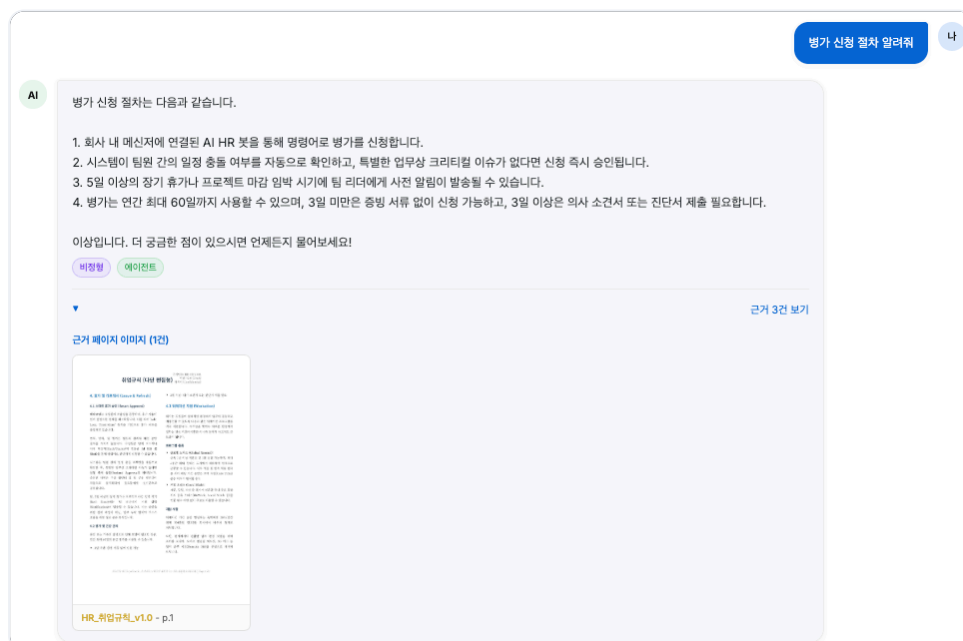


그림 11-4. 병가 규정이 정확히 돌아옵니다. 출장·급여 같은 다른 주제는 섞이지 않습니다

Semantic 청킹이 병가 조항만 한 덩어리로 유지하고, 리랭커가 관련 없는 청크를 밀어낸 결과입니다. 그런데 눈에 보이는 답변만으로는 내부에서 뭐가 일어났는지 알 수 없습니다. 오픈이가 서버를 돌리고 있는 터미널 창으로 시선을 돌렸습니다.

```

ex11 - python run.py · POST /api/chat

질문 수신 | 병가 신청 절차 알려줘

QueryExpander 병가→병가(병가 유급휴가)
원본: 병가 신청 절차 알려줘
확장: 병가(병가 유급휴가) 신청 절차 알려줘

TokenTracker 기록
모델 llama3.1:8b
입력 토큰 6
출력 토큰 226
비용 (USD) $0.000000
지연 시간 24993 ms

처리 완료 | 유형=unstructured · 소요=24993ms
    
```

그림 11-5. 병가 질문이 들어온 순간부터 응답이 나갈 때까지의 풀 로그

사전에 등록된 **병가**가 **병가(병가 유급휴가)**로 풀어져 검색으로 넘어갔습니다. 이어서 llama3.1 모델이 검색 결과로 답을 만들어 내는 데 24.9초가 걸렸고, 유형은 비정형(**unstructured**)으로 기록됩니다. 챗터 7에서 붙인 TokenTracker가 챗터 11에서도 그대로 일하고 있습니다.

다음은 약어가 섞인 질문. **WFH 정책 알려줘**.

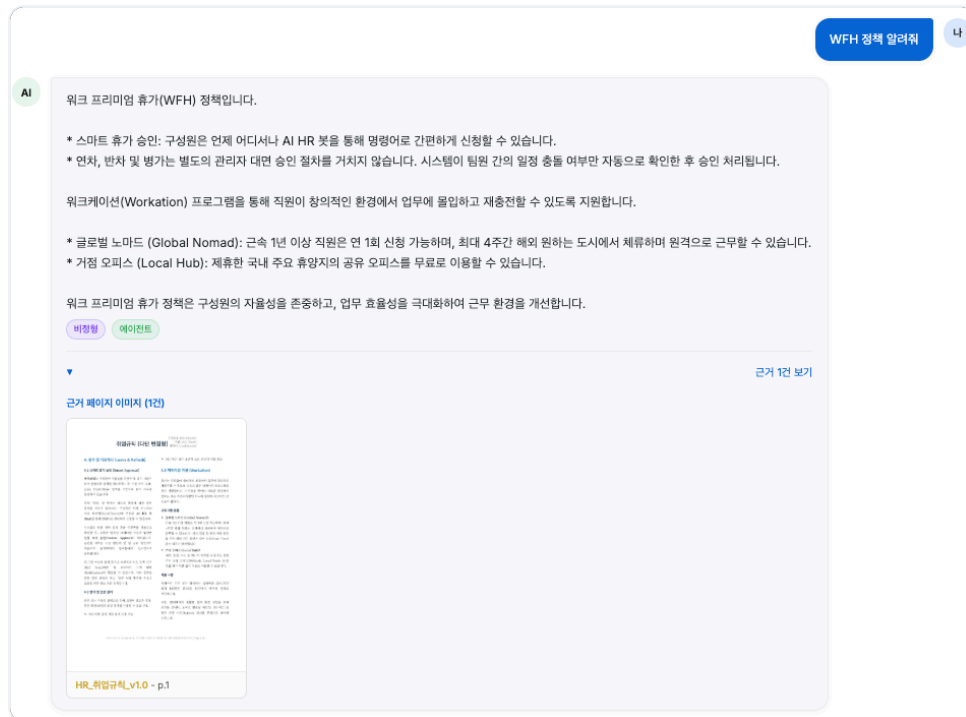


그림 11-6. 취업규칙의 재택·원격 근무 조항이 근거로 돌아옵니다

답은 나왔는데, 이번엔 조금 더 흥미로운 구석이 있습니다. 취업규칙 문서에는 "재택근무"라는 단어가 없고 "원격 근무"만 쓰여 있는데, 답변은 정확히 찾아왔습니다. 오픈이가 다시 터미널을 봤습니다.



그림 11-7. WFH가 재택근무로 확장된 뒤 동일 경로를 밟습니다

사전이 **WFH** 에 **(재택근무)** 를 덧붙였습니다. 그런데 이 확장어는 문서에 없는 표현입니다. 그 간극을 벡터 임베딩이 메워 줍니다. **재택근무** 와 **원격 근무** 는 의미 공간에서 가까운 자리에 놓여 있어 한쪽을 질문으로 들이밀면 다른 쪽 문서가 걸립니다. 챗터 9의 약어 사전이 못 덮는 표현 차이를 챗터 4의 임베딩 모델이 메우는 협업입니다. 출력 토큰이 226→238로 조금 늘어난 것은 LLM이 워케이션·거점 오피스 두 항목을 합쳐 답했기 때문입니다.

마지막 질문. **정보보안서약서 내용이 뭐야?**

정보보안서약서 내용이 뭐야?
나

AI

정보보안 서약서는 다음과 같은 내용을 포함합니다.

1. 기본 원칙: 회사는 업무 생산성을 위해 승인된 AI 도구의 사용을 장려한다. 단, 정보 자산의 유출을 방지하기 위해 본 지침을 엄격히 준수해야 한다.
2. 금지 사항: 고객 개인정보(PII), 핵심 소스코드, 미공개 전략 문서는 어떠한 경우에도 외부 퍼블릭 AI에 입력해서는 안 된다.
3. 사용 가능 도구: 회사가 라이선스를 제공하는 엔터프라이즈급 AI 도구만 사용이 허가된다.
4. 보안 설정 의무: AI 도구를 사용할 때는 반드시 데이터 학습 방지 기능인 'Enterprise Shield' 모드(또는 Opt-out 설정)가 활성화되어 있는지 확인해야 한다.

AI가 생성한 코드 및 문서에 대한 최종 책임은 사용자 본인에게 있다. AI의 오류(Hallucination)로 인한 서비스 장애 발생 시, 이를 검증하지 않은 사용자가 책임을 진다.

이 정보는 회사 내부에서만 유효하고, 대외비로 분류되어 있습니다.

비정형
에이전트

[근거 2건 보기](#)

▼
근거 페이지 이미지 (1건)

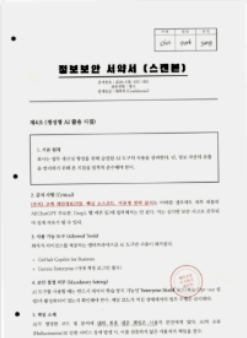


그림 11-8. 제4조 생성형 AI 활용 지침까지 조항 번호가 또렷하게 돌아옵니다

한 박자 기다렸습니다. 노트북 팬이 숨을 한 번 몰아쉬고 서약서 본문이 그대로 돌아왔습니다. 제4조, 제5조, 조항 번호까지 정확합니다. 터미널에는 이런 줄이 찍혔습니다.



그림 11-9. 약어가 없는 질문은 원본 쿼리가 그대로 흐릅니다. 지연은 늘어납니다

이번엔 사전에 등록된 단어가 없어 원본이 그대로 검색으로 내려갔습니다. 그래도 답이 또렷한 이유는 인덱싱 단계에 있습니다. 챗터 10에서 붙인 Vision 파서가 스캔본 한 장을 텍스트로 꺼내 벡터DB에 올려 뒀기 때문입니다. 지연 시간이 24초 → 36초로 늘어난 것도 눈에 띕니다. 스캔본에서 꺼낸 내용이 구조가 복잡한 표·조항 나열이라 LLM이 답을 엮는 데 시간이 더 걸립니다. 약어 확장이 아니라 인덱싱 때의 투자가 결과를 살린 케이스입니다.

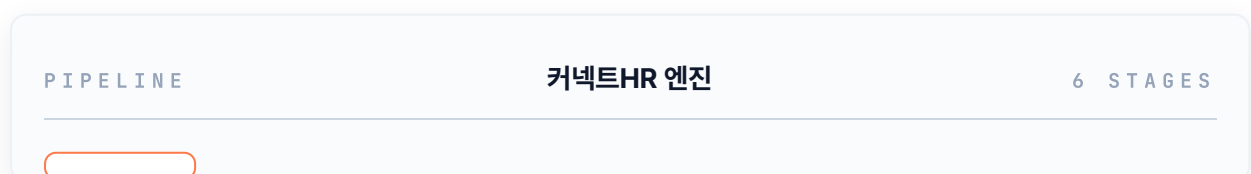
세 개 다 잡았네.

오픈이 : "병가·WFH·서약서 다 되네요. 서약서는 챗터 10에서 Vision 붙여 둔 덕분이네요."

동료 : "질문 타입이 세 개 다 다른데 세 번 다 근거까지 붙네요."

11.4 엔진의 내부 - 파이프라인 한 장

웹 화면에서 본 세 답변이 안에서는 어떤 단계를 거쳐 만들어졌는지, 파이프라인 전체를 한 장에 정리하면 이렇게 보입니다.



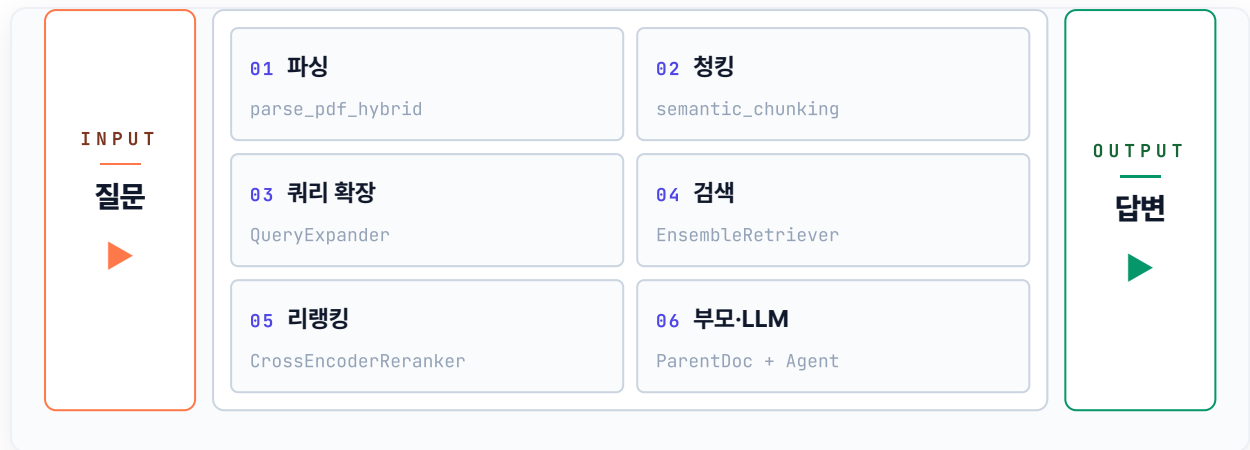


그림 11-10. 질문이 들어와 6단계 파이프라인을 지나 답변이 나가기까지

01(파싱)과 02(청킹)는 서버가 처음 뜰 때 한 번만 도는 **인덱싱** 단계입니다. 문서를 벡터DB에 올리는 일이라 매 질문마다 반복할 필요가 없습니다. 나머지 03부터 06(쿼리 확장·검색·리랭킹·부모·LLM)이 질문이 들어올 때마다 도는 **질의** 단계이고, 실습 1의 `run_pipeline` 이 이 03~06을 차례로 호출합니다.

챗터 8~10에서 따로 실험하던 부품들이 각 단계에 박혀 있습니다. Semantic 청킹은 02, 약어 사전은 03, 하이브리드 검색은 04, 리랭커는 05, 부모 문서 확장과 LLM 답변 생성은 06. 따로 볼 때는 각자의 평가표만 있었지만, 지금은 한 줄로 꿰어져 한 질문의 응답을 함께 만들어 냅니다.

11.5 전체 구성도의 완성

11.4의 파이프라인이 "질문이 답변으로 흐르는 길"이었다면, 아래 구성도는 그 길을 떠받치는 **시스템 전체**를 한 장에 담은 모습입니다. LLM부터 에이전트 루프, 도구, 저장소까지.



그림 11-11. 커넥트HR 에이전트의 시스템 아키텍처. 사용자부터 LLM, 에이전트 루프, 도구, 저장소까지 한 장에 담았습니다

상단 Ollama가 프롬프트를 받아 답변을 돌려주고, 통합 에이전트 안의 ReAct 루프가 4개 도구를 선택적으로 호출해 PostgreSQL/ChromaDB에서 데이터를 꺼냅니다.

오픈이가 모니터에서 눈을 뗐습니다. 창가 쪽 형광등이 한 번 깜빡였고, 키보드 두드리는 소리가 멀리서 작게 들렸습니다.

자리 뒤로 동료 몇 명이 서 있었습니다. 누군가는 채팅창에, 누군가는 구성도 한 장에 눈이 가 있었습니다.

동료 : "병가·WFH·서약서 다 잡네요. 고생했어요."

조각들이 한 장에 붙어 버렸네.

챗터 1의 맛보기 RAG에서 출발해 여기까지 왔습니다. 문서 규칙을 세우고(챗터 2~3), 벡터로 옮기고(챗터 4), RAG 엔진을 엮고(챗터 5), 통합 에이전트로 다시 짜고(챗터 6), 운영 안정화(챗터 7), 검색 품질(챗터 8), 쿼리 해석(챗터 9), 멀티모달(챗터 10), 그리고 평가와 조립(챗터 11)까지. 그사이 **커넥트HR 에이전트** 한 명이 완성됐습니다.

팀장이 한참 뒤에 지나가다 모니터를 들여다봤습니다.

팀장 : "이론 안 엮고 붙이기만 한 거, 이게 이번에 가장 어려운 일이었을 거예요."

책은 여기서 끝나지만, 시스템은 출발선입니다. 오늘 만든 엔진이 내일 어떤 질문에서 무너지는지는 이제 사무실 바깥의 사용자가 알려 줄 겁니다.

용어 정리

본문 속 표현	진짜 용어	정식 정의
"한 줄로 엮은 조립"	RAG Pipeline	파싱→청킹→검색→리랭킹→쿼리 재작성→LLM 답변까지 하나의 데이터 흐름으로 이어붙인 구조
"답변 옆의 증거"	Evidence / Citation	답변 근거가 된 원본 문서 조각. 텍스트 인용·링크·이미지 캡처 등으로 제공
"정형 / 비정형 분기"	Response Modality Routing	질문 성격에 따라 응답 형태(JSON 수치·문서 이미지 등)를 달리 내보내는 로직

"세 층으로 나눈 구조"	레이어드 아키텍처 (Layered Architecture)	Application → Domain → Infrastructure 세 층이 단방향으로만 호출하는 코드 구조 패턴
"필요한 부품을 처음에 받아 두는 방식"	생성자 주입 (Constructor Injection)	객체가 의존 부품을 직접 만들지 않고 외부에서 주입받는 패턴. 객체 수명·테스트 용이성·교체 비용에 영향
"챗터 10 메뉴판의 네 조합 중 D"	D Configuration (Full Stack)	Semantic 청킹 + 하이브리드 검색 + 리랭킹 + 약어 확장 + Parent Doc + Vision 파서가 모두 켜진 조합

이것만은 기억하자

- **커넥트HR이라는 사서 한 명이 완성됐습니다.** 챗터 4에서 손에 자료(임베딩)를 쥐여 준 뒤 챗터 8에서 면접관(리랭커)을, 챗터 9에서 귀(약어 사전)를, 챗터 10에서 눈(Vision)을 차례로 달았고, 이번 챗터에서 그 부품들을 한 파일에 묶어 한 명의 직원으로 출근시켰습니다.
- **조립이 가장 큰 일의 절반입니다.** 챗터 8~10에서 실험한 튜닝 부품을 한 파일에 함수로 모아 `run_pipeline` 하나로 엮으면 파이프라인이 완성됐습니다. 쌓인 부품이 많을수록 조립의 힘이 커집니다.
- **근거는 답변만큼 중요합니다.** 사용자는 답의 내용뿐 아니라 "어디서 나왔는지"로 신뢰를 판단합니다. 정형 수치는 JSON으로, 비정형 내용은 문서 이미지로. 질문의 성격에 맞춰 근거의 모양을 바꿉니다.
- **여기서 끝이 아닙니다.** 평가셋을 늘리고, 사용자 로그로 실패 쿼리를 수집하고, Langfuse 같은 도구로 운영 중 품질을 상시 관측하세요. 커넥트HR이라는 사서는 이제 **계속 자라는** 시스템입니다.